

Projects - Simplified Story and Deep-Dive Defence

What to say, what to hold back, and how to survive a schema-and-SQL ambush

Nirmalya Mandal - BT Group Interview Prep

Study pack - part 3 of 8

Contents

The lesson from SAP	3
The rules for talking about projects tomorrow	4
Project 1: ChatterBox - lead with this	5
The 30-second pitch	5
The “tricky bit” line, if you want to show depth	5
Cross-questions and answers	5
The prepared route and payload	6
The prepared schema	6
The SQL they will ask off that schema	7
Project 2: FloatChat	9
The 30-second pitch (deliberately small)	9
If they push	9
Prepared route and payload	9
Prepared schema - three tables only	9
Likely SQL off it	10
Project 3: AUKTAVE 2K26	11
The 30-second pitch	11
Prepared route and payload	11
Prepared schema	11
Likely SQL	12
Experience 1: Modern Mahal (Jun - Jul 2026)	13
The 30-second pitch	13
The part you actually want them to ask about	13
Prepared route and payload	13
Prepared schema	14
Likely SQL	14
Experience 2: DokLink (Jun - Aug 2025, internship)	15
The 30-second pitch	15
If they push	15
Prepared route and payload	15
Prepared schema	15
Likely SQL	16
The SQL revision card	17
The two-minute drill - do this before you sleep	18

The lesson from SAP

At SAP they did not ask about your projects in the abstract. They:

1. Asked you to **write out any route of the backend and show the payload for that route.**
2. Asked you to **draw the main tables of the project.**
3. Then asked **SQL questions based on the schema you had just drawn.**

That is a deep-dive ambush, and it will happen again - at BT, or at the next company. Round 2 here is explicitly “networking + projects from CVs”.

The defence is not to know more. It is to have already decided, in advance, exactly which route, which schema, and which tables you will produce. Then the ambush lands on ground you chose.

This document gives you, for every project on your CV: the simplified pitch, the deeper layer to reveal only if pushed, one prepared route with its payload, one prepared schema, and the SQL that follows from it.

The rules for talking about projects tomorrow

Rule 1 - describe them at the size they are written on your CV. Your CV now says small, simple things. Say small, simple things. Simple descriptions invite simple follow-ups, and simple follow-ups you can answer completely.

Rule 2 - simplified is fine, wrong is fatal. You may leave things out. You may say “it’s a small project”. You may not state a technical fact that is untrue. If they push into a corner, tell the truth about that corner.

Rule 3 - lead them to ChatterBox. It is the project where the follow-up questions are *networking* questions, which is the ground you have prepared. When asked “tell me about a project”, start there.

Rule 4 - answer for 30 seconds, then stop. Silence after your answer is not your problem to fill. They will ask the next question.

Rule 5 - one prepared depth line per project. If a round is going well and you want to show you are more than the CV, use the “the tricky bit was...” line given for each project below. One, not three.

Project 1: ChatterBox - lead with this

Full technical spec is in **document 04**. This section is how you *talk* about it.

The 30-second pitch

“It’s a small chat application - two users can send messages to each other and see them appear instantly without refreshing. The part I found interesting is that normally the web is request-response: the browser asks, the server answers, connection closed. That doesn’t work for chat, because the server needs to push a message to you when someone else sends it. So I used WebSockets, where one TCP connection stays open and either side can send whenever they want.”

Then **stop**. That answer hands them three doors, all of which you want them to open: WebSockets, TCP, and push-versus-poll.

The “tricky bit” line, if you want to show depth

“The bit that took me longest was handling a connection dropping. If someone’s Wi-Fi dies, the TCP socket doesn’t always close cleanly - the server can still think they’re connected. So I added a heartbeat: the server pings every 30 seconds and if there’s no pong back, it marks the user offline and cleans up. That was the first time I really understood that a connection being ‘open’ is partly an assumption.”

That is a networking answer wearing a project costume. It is the best sentence you have for this interview.

Cross-questions and answers

“Why WebSockets and not just refreshing / polling?” > “Polling means the browser asks ‘anything new?’ every couple of seconds. Most of those requests come back empty, so you’re wasting requests and you still get a delay. WebSockets keep one connection open, so the server sends the message the moment it arrives - less traffic and no delay.”

“How does a WebSocket connection start?” > “It starts as a normal HTTP request with an `Upgrade: websocket` header. The server replies with `101 Switching Protocols`, and from then on the same TCP connection is used for WebSocket frames instead of HTTP. So it’s HTTP for the handshake, then it stops being HTTP.”

“Does it use TCP or UDP? Why?” > “TCP. Messages have to arrive, and arrive in order - if you lose a message in a chat, or the messages arrive out of order, the app is broken. UDP would be faster but gives no guarantee of either. UDP makes sense for voice or video where a lost packet is better than a late one, but not for text messages.”

“What port does it run on?” > “In development, 3000. In production it would sit behind Nginx on 443, with `wss://` - which is WebSocket over TLS, the same way HTTPS is HTTP over TLS.”

“What happens if the server restarts?” > “All the open connections drop, and the clients have to reconnect. My client retries automatically after a short delay. The messages themselves are safe

because they're in PostgreSQL, not held in memory."

"How would you scale it to a million users?" > The humble, correct answer: "Honestly, it doesn't scale as written, because the open connections live in one server's memory. If you ran two servers, a user connected to server A wouldn't get a message sent through server B. The usual fix is a shared pub/sub layer like Redis so all the servers hear every message. I know the shape of the solution but I haven't built it."

That answer is better than a confident wrong one. It shows you understand the limit of your own work.

The prepared route and payload

If they say "write any route and show me the payload", **write this one**. It is small, it is complete, and every field is defensible.

POST /api/messages

Request headers:

Content-Type: application/json

Authorization: Bearer <jwt token>

Request body:

```
{
  "chat_id": 12,
  "content": "Are we still meeting at 5?"
}
```

Response - 201 Created:

```
{
  "id": 348,
  "chat_id": 12,
  "sender_id": 3,
  "content": "Are we still meeting at 5?",
  "created_at": "2026-08-13T09:15:22Z",
  "status": "sent"
}
```

What to say while you write it: "The client sends only the chat and the text. The sender comes from the token, not the body - if I trusted a `sender_id` from the client, anyone could post as anyone else. The server fills in the id, the timestamp and the status."

That one sentence about not trusting the client's `sender_id` is worth more than the whole route. It shows security thinking without you claiming to be a security expert.

The prepared schema

Draw exactly these four boxes. Do not add more.

users		chats		chat_members		messages	
-----		-----		-----		-----	
id	PK	id	PK	id	PK	id	PK
username	UNIQUE	name		chat_id	FK -> chats	id	
password_hash		is_group		user_id	FK -> users	chat_id	FK -> chats.id
is_online	BOOL	created_at		joined_at		sender_id	FK -> users.id
last_seen						content	
created_at						created_at	

Say while drawing: “users and chats are the two main things. `chat_members` is the join table, because a chat has many users and a user is in many chats - that’s a many-to-many, so it needs its own table. Messages belong to one chat and one sender.”

If they ask why `chat_members` exists - that is the question they *want* to ask, so have the answer ready: “Because it’s many-to-many. I can’t put a user id on the chat, since a chat has several users, and I can’t put chats on the user for the same reason. The join table holds one row per membership.”

The SQL they will ask off that schema

Practise writing these by hand. Neat, on paper, saying what you are doing.

1. All messages in chat 12, newest last:

```
SELECT * FROM messages WHERE chat_id = 12 ORDER BY created_at ASC;
```

2. Messages with the sender’s username (the join question - they always ask a join):

```
SELECT m.content, u.username, m.created_at
FROM messages m
JOIN users u ON u.id = m.sender_id
WHERE m.chat_id = 12
ORDER BY m.created_at;
```

3. Number of messages each user has sent (the GROUP BY question):

```
SELECT u.username, COUNT(m.id) AS message_count
FROM users u
LEFT JOIN messages m ON m.sender_id = u.id
GROUP BY u.username
ORDER BY message_count DESC;
```

Say why LEFT JOIN: “So users who have never sent a message still appear, with a count of zero. An inner join would drop them.”

4. The most recent message in every chat (the hard one - be ready):

```
SELECT DISTINCT ON (chat_id) chat_id, content, created_at
FROM messages
ORDER BY chat_id, created_at DESC;
```

*If you blank on **DISTINCT ON**, the portable version is fine and they will accept it:*

```
SELECT m.* FROM messages m
JOIN (SELECT chat_id, MAX(created_at) AS latest
      FROM messages GROUP BY chat_id) x
ON x.chat_id = m.chat_id AND x.latest = m.created_at;
```

5. All chats a given user belongs to:

```
SELECT c.* FROM chats c
JOIN chat_members cm ON cm.chat_id = c.id
WHERE cm.user_id = 3;
```

6. Users currently online:

```
SELECT username, last_seen FROM users WHERE is_online = TRUE;
```

If they ask about indexing - a very likely follow-up: “I’d index `messages.chat_id` and `messages.created_at`, because the query that runs constantly is ‘give me this chat’s recent messages’. Without an index the database scans every message row. The cost is that writes get slightly slower and the index takes storage.”

Project 2: FloatChat

The 30-second pitch (deliberately small)

“It’s a web app for ocean sensor data. There’s a global set of floating sensors that record things like temperature and salinity at different depths. Normally you’d need to write a database query to get anything out of it. In this, you type the question in plain English - like ‘show salinity near the equator in 2015’ - and it turns that into an SQL query, runs it, and draws the result as a table and a map.”

Do not volunteer: LLM pipelines, RAG, vector databases, embeddings, ETL, 3D globes, Mistral 7B. Not because they are untrue, but because each one invites twenty minutes of questioning that is not what tomorrow is about.

If they push

“**How does it turn English into SQL?**” > “It sends the question to a language model along with a description of the database tables, and the model produces the SQL. Then I validate the query before running it - checking it’s a read-only SELECT and nothing else - because you can’t just execute whatever a model gives you.”

That validation point is the honest, sensible-engineer answer. Stop there.

“**What if it generates a wrong query?**” > “Then you get a wrong answer, which is the real weakness of the approach. I limited the damage by restricting it to read-only queries and showing the user the query that ran, so it’s visible rather than hidden.”

Prepared route and payload

POST /api/query

```
{ "question": "average temperature near the equator in 2015" }
```

Response - 200 OK:

```
{
  "sql": "SELECT AVG(temperature) FROM measurements WHERE latitude BETWEEN -5 AND 5
    ↪ AND EXTRACT(YEAR FROM measured_at) = 2015",
  "columns": ["avg_temperature"],
  "rows": [[27.4]],
  "row_count": 1
}
```

Prepared schema - three tables only

floats		profiles		measurements	
-----		-----		-----	
id	PK	id	PK	id	PK
platform_no	UNIQUE	float_id	FK->floats	profile_id	FK->profiles

deployed_at	latitude	depth
status	longitude	temperature
	measured_at	salinity

“A float is one physical sensor. Every time it surfaces it records a profile - a position and a time. Each profile has many measurements at different depths.”

Likely SQL off it

```
-- average temperature near the equator in 2015
SELECT AVG(m.temperature)
FROM measurements m
JOIN profiles p ON p.id = m.profile_id
WHERE p.latitude BETWEEN -5 AND 5
      AND EXTRACT(YEAR FROM p.measured_at) = 2015;

-- how many profiles each float has recorded
SELECT f.platform_no, COUNT(p.id) AS profile_count
FROM floats f
LEFT JOIN profiles p ON p.float_id = f.id
GROUP BY f.platform_no
ORDER BY profile_count DESC;

-- the deepest measurement recorded by each float
SELECT f.platform_no, MAX(m.depth) AS max_depth
FROM floats f
JOIN profiles p ON p.float_id = f.id
JOIN measurements m ON m.profile_id = p.id
GROUP BY f.platform_no;
```

Project 3: AUKTAVE 2K26

The 30-second pitch

“The website for my university’s tech fest - event pages, the schedule, and a registration form. Built with Next.js. Nothing complicated, but it had to actually stay up during the fest, and the traffic came in short bursts whenever an event was announced.”

That last sentence is a deliberate hook: it is a **reliability** sentence, on a CV for a reliability role.

If they take the hook - “what did you do about the bursts?” > “Most of the pages are static, so they’re served as pre-built files rather than being generated per request, which handles bursts well. The registration form was the part that actually hit the database, so that’s where the load went. It held up, but I was watching the logs during the big announcements.”

Prepared route and payload

POST /api/register

```
{
  "event_id": 7,
  "name": "Nirmalya Mandal",
  "email": "nirmalya@example.com",
  "phone": "7001467098",
  "college": "Amity University Kolkata"
}
```

Response - 201 Created:

```
{ "registration_id": 214, "event_id": 7, "status": "confirmed" }
```

Prepared schema

events		registrations	
-----		-----	
id	PK	id	PK
title		event_id	FK -> events.id
description		name	
venue		email	
starts_at		phone	
capacity		college	
		created_at	
		UNIQUE (event_id, email)	

The unique constraint is the detail worth mentioning: “I put a unique constraint on event plus email so the same person can’t register twice for the same event. Doing it in the database rather than only in the code means it holds even if two requests arrive at the same moment.”

That is a race-condition answer in one plain sentence - which is exactly the right size for this interview.

Likely SQL

```
-- registrations per event
SELECT e.title, COUNT(r.id) AS total
FROM events e LEFT JOIN registrations r ON r.event_id = e.id
GROUP BY e.title ORDER BY total DESC;

-- events that are full
SELECT e.title, e.capacity, COUNT(r.id) AS registered
FROM events e JOIN registrations r ON r.event_id = e.id
GROUP BY e.id, e.title, e.capacity
HAVING COUNT(r.id) >= e.capacity;
```

Know the difference: **WHERE** filters rows before grouping, **HAVING** filters after grouping. That is a classic follow-up.

Experience 1: Modern Mahal (Jun - Jul 2026)

The 30-second pitch

“A small internal web tool for a glass business. They were tracking customer orders on paper, so I built something where staff enter an order and then move it through the production stages, and there’s a page that generates a price quotation. Two access levels - staff and admin. Then I put it on a Linux server for them.”

The part you actually want them to ask about

“Setting it up on the server was the part I learned most from - Nginx in front as a reverse proxy, DNS records pointed at the server’s IP, and a Let’s Encrypt certificate for HTTPS. The application itself runs on a local port and only Nginx is exposed to the internet.”

Follow-ups you should be ready for - these are networking questions:

“**Why put Nginx in front at all?**” > “So the application isn’t directly exposed. Nginx handles TLS, serves static files efficiently, and forwards the rest to the app on localhost. It also means I can change what’s running behind it without changing anything public.”

“**Which DNS records did you add?**” > “An A record pointing the domain to the server’s IPv4 address, and a CNAME for `www` pointing to the root domain.”

“**How does the HTTPS certificate work?**” > “Let’s Encrypt issues a certificate after verifying I control the domain - it gives a challenge file that has to appear at a specific URL on the site. The certificate is then presented during the TLS handshake, and the browser trusts it because Let’s Encrypt is a trusted certificate authority. They expire every 90 days, so renewal is automated.”

“**Which ports were open?**” > “22 for SSH, 80 and 443 for the web. Everything else closed at the firewall. Port 80 mainly redirects to 443.”

This block is the strongest fit-for-role material you have. Know it cold.

Prepared route and payload

POST /api/orders

```
{
  "customer_id": 5,
  "items": [
    { "product": "Toughened glass 8mm", "width_mm": 1200, "height_mm": 800,
      ↪  "quantity": 4 }
  ],
  "delivery_date": "2026-07-20"
}
```

Response - 201 Created:

```
{ "order_id": 91, "status": "pending", "total_amount": 18400, "created_at":
  ↪ "2026-07-02T11:04:00Z" }
```

Prepared schema

customers	orders	order_items	users
-----	-----	-----	-----
id PK	id PK	id PK	id PK
name	customer_id FK	order_id FK->orders	username
phone	status	product	password_hash
address	total_amount	width_mm	role (staff admin)
created_at	delivery_date	height_mm	
	created_at	quantity	
		unit_price	

Likely SQL

```
-- total value of orders per customer
SELECT c.name, SUM(o.total_amount) AS total_spent
FROM customers c JOIN orders o ON o.customer_id = c.id
GROUP BY c.name ORDER BY total_spent DESC;

-- orders still pending, oldest first
SELECT id, customer_id, created_at FROM orders
WHERE status = 'pending' ORDER BY created_at ASC;

-- number of items in each order
SELECT o.id, COUNT(oi.id) AS item_count
FROM orders o LEFT JOIN order_items oi ON oi.order_id = o.id
GROUP BY o.id;
```

Experience 2: DokLink (Jun - Aug 2025, internship)

Your CV now says Full Stack Developer Intern. Talk like an intern. Say “I worked on”, “I helped build”, “I was shown”. Do not say you owned it, ran it, or decided it - the paper in front of them says otherwise, and a mismatch is the one thing that damages you.

The 30-second pitch

“It was a healthcare app - you open it and it shows nearby hospitals with how many beds are free, and you can request one. I worked on the backend APIs, mainly login, the hospital listing and the bed booking endpoint, in Django with PostgreSQL. I also got to see how the whole thing was deployed and run, which was new to me at the time.”

If they push

“What was the hardest part?” > “Making sure two people couldn’t book the same last bed at the same time. If you just read the count and then decrement it, two requests can both read ‘one left’ and both succeed. It has to be done inside a database transaction that locks the row, so the second one waits and then sees the real number.”

That is your strongest technical story and it survives being an intern’s story perfectly - you were shown a real problem and you understood it.

“What did you learn?” > “That making something work on your own laptop and making it work reliably for real users are completely different jobs. Most of what I learned there was about the second one - deployment, logs, what happens when something fails at 11pm.”

Prepared route and payload

POST /api/bookings

```
{ "hospital_id": 14, "bed_type": "icu", "patient_name": "R. Sharma" }
```

Response - 201 Created:

```
{
  "booking_id": 502,
  "hospital_id": 14,
  "bed_type": "icu",
  "status": "reserved",
  "expires_at": "2026-08-13T10:45:00Z"
}
```

Worth saying: “The reservation has an expiry, so if the patient never arrives the bed goes back into the pool automatically instead of being held forever.”

Prepared schema

users	hospitals	beds	bookings
-------	-----------	------	----------

-----		-----		----		-----	
id	PK	id	PK	id	PK	id	PK
name		name		hospital_id	FK	user_id	FK->users
phone		address		bed_type		bed_id	FK->beds
role		latitude		is_occupied		status	
created_at		longitude				created_at	
		contact_number				expires_at	

Likely SQL

```

-- free ICU beds per hospital
SELECT h.name, COUNT(b.id) AS free_icu
FROM hospitals h JOIN beds b ON b.hospital_id = h.id
WHERE b.bed_type = 'icu' AND b.is_occupied = FALSE
GROUP BY h.name;

-- all active bookings with the hospital name
SELECT bk.id, u.name AS patient, h.name AS hospital, bk.status
FROM bookings bk
JOIN users u ON u.id = bk.user_id
JOIN beds b ON b.id = bk.bed_id
JOIN hospitals h ON h.id = b.hospital_id
WHERE bk.status = 'reserved';

-- bookings that have expired and should be released
SELECT id, bed_id FROM bookings
WHERE status = 'reserved' AND expires_at < NOW();

```

The SQL revision card

If they ask SQL off any schema, it will be one of these six shapes. Know them, not the specific queries.

Shape	Pattern
Filter	<code>SELECT ... WHERE ... ORDER BY ...</code>
Join two tables	<code>JOIN other ON other.id = this.other_id</code>
Count per group	<code>SELECT x, COUNT(*) FROM t GROUP BY x</code>
Filter after grouping	<code>GROUP BY ... HAVING COUNT(*) > n</code>
Include the empty ones	<code>LEFT JOIN</code> instead of <code>JOIN</code>
Top / latest per group	<code>MAX()</code> in a subquery joined back, or <code>DISTINCT ON</code>

Definitions they may ask alongside:

- **Primary key** - uniquely identifies a row; not null, one per table.
- **Foreign key** - a column pointing at another table's primary key; enforces that the referenced row exists.
- **INNER vs LEFT JOIN** - inner keeps only matching rows on both sides; left keeps all rows from the left table, with nulls where there is no match.
- **WHERE vs HAVING** - before grouping vs after grouping.
- **Index** - a structure (usually a B-tree) that speeds up lookups; costs storage and slows writes.
- **Transaction / ACID** - Atomicity (all or nothing), Consistency, Isolation (concurrent transactions don't interfere), Durability (committed data survives a crash).
- **Normalisation** - organising tables to avoid duplicating data; 1NF atomic values, 2NF no partial dependency on part of a composite key, 3NF no non-key column depending on another non-key column.

The two-minute drill - do this before you sleep

On a blank sheet of paper, without looking:

1. Write the `POST /api/messages` route with its request and response payload.
2. Draw the four ChatterBox tables with their keys.
3. Write the join query for messages with usernames.
4. Say the ChatterBox 30-second pitch out loud.

If you can do those four things, the round-2 ambush cannot land.